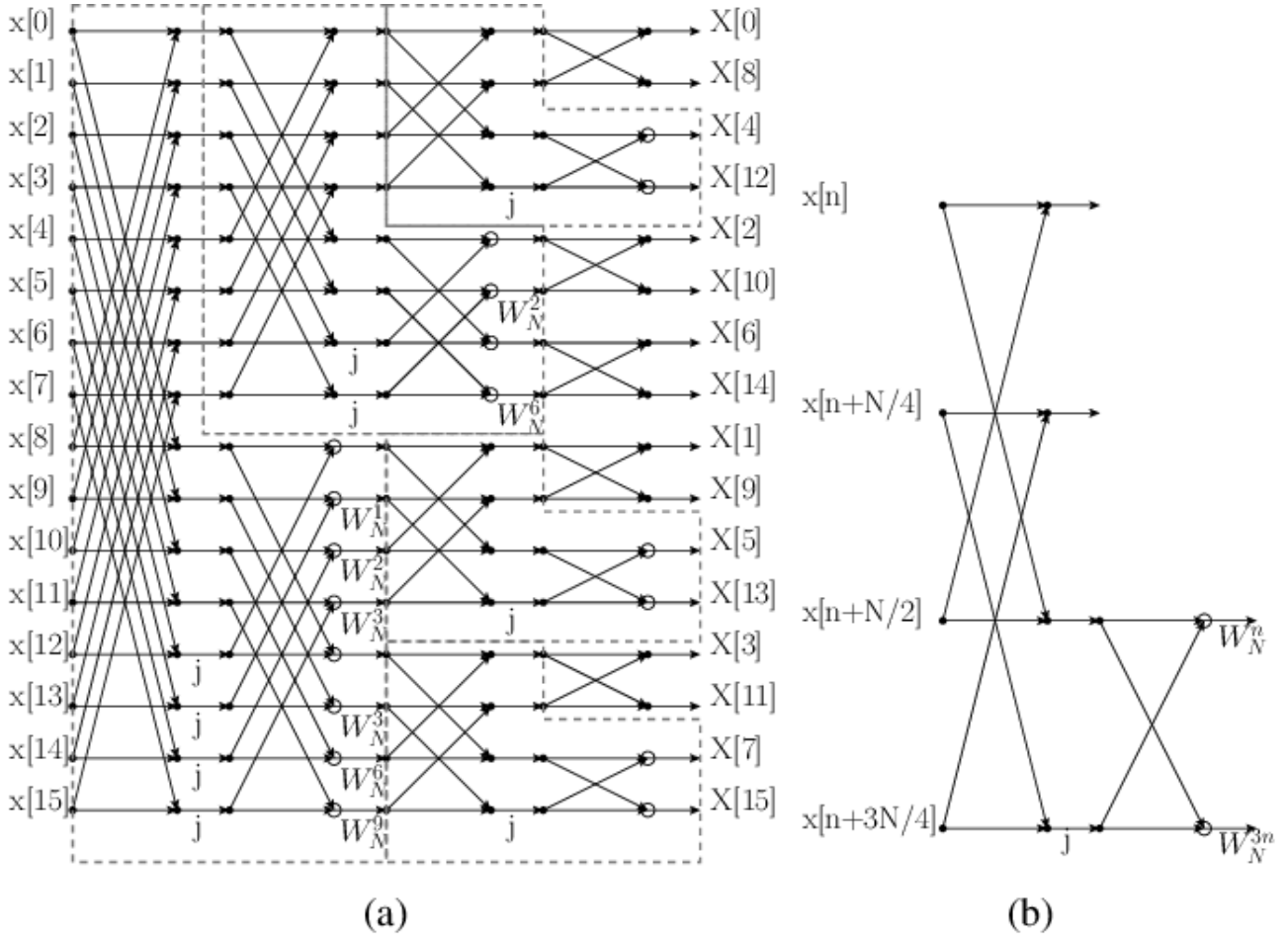
FPGA-Based Hardware Implementation of a Scalable, Streaming Split-Radix FFT Processor

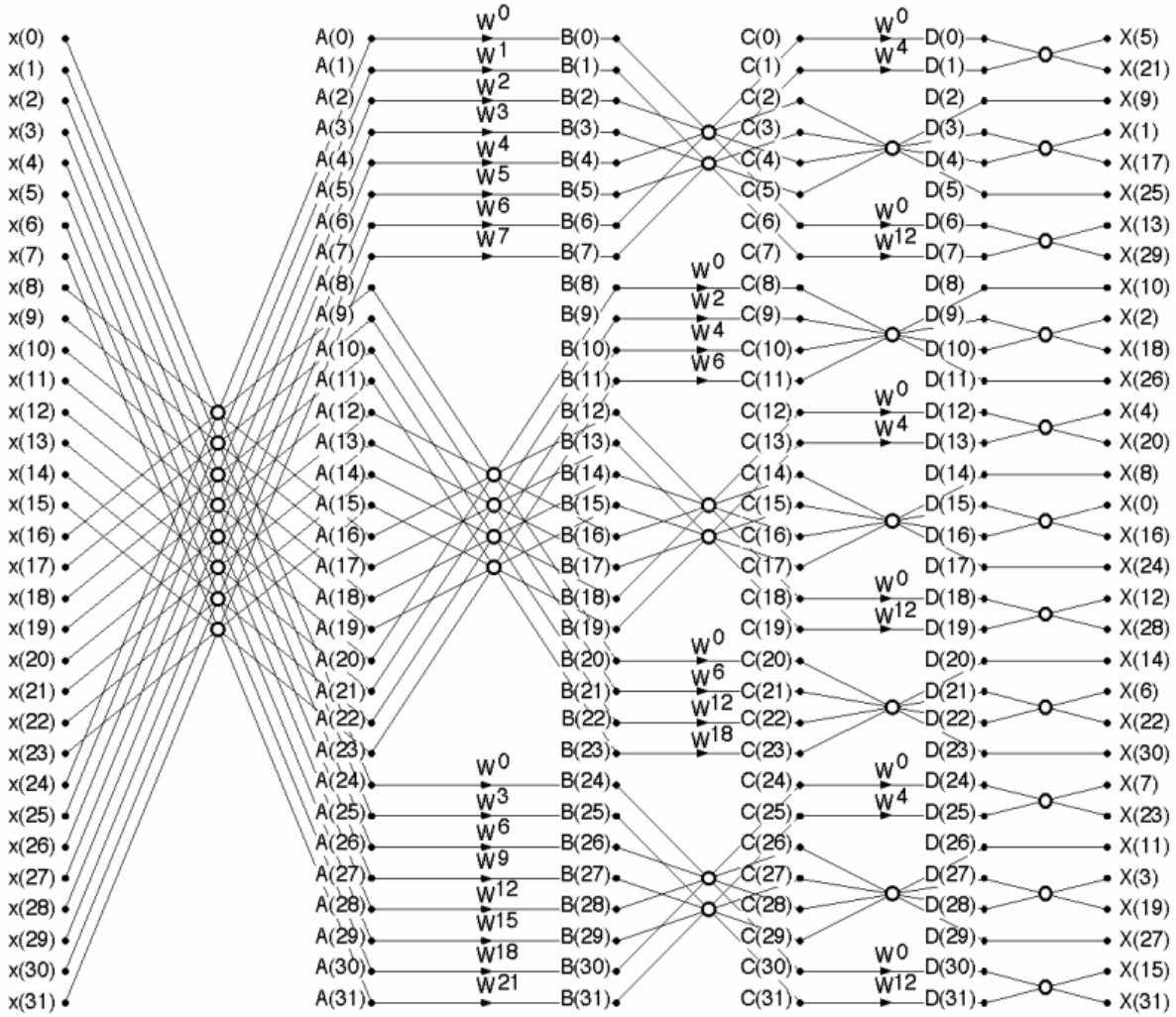
Christodoulos Zerdalis
03531

Classic Split-Radix



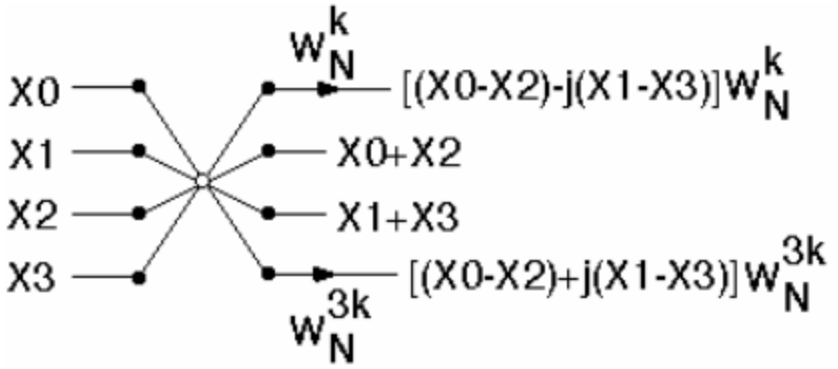
- **L-shaped** geometry
- Very **irregular pattern** for an **SDF** implementation
- **Butterfly** supports only **two samples** per clock

Revised Split-Radix



- **Symmetrical** geometry **down the middle**
- More **regular structure** that supports **four samples** per clock.

Revised Butterfly



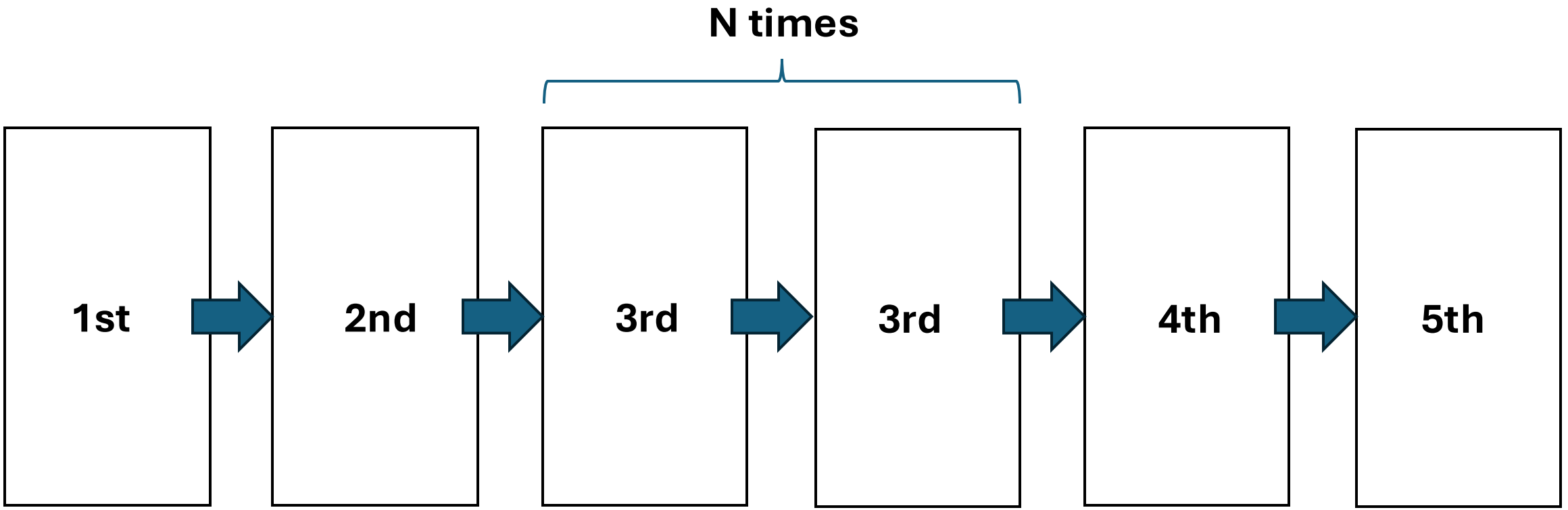
- **Four samples** per iteration
- Does **not** use **multiplication,** these **happen independently**

Hardware Design

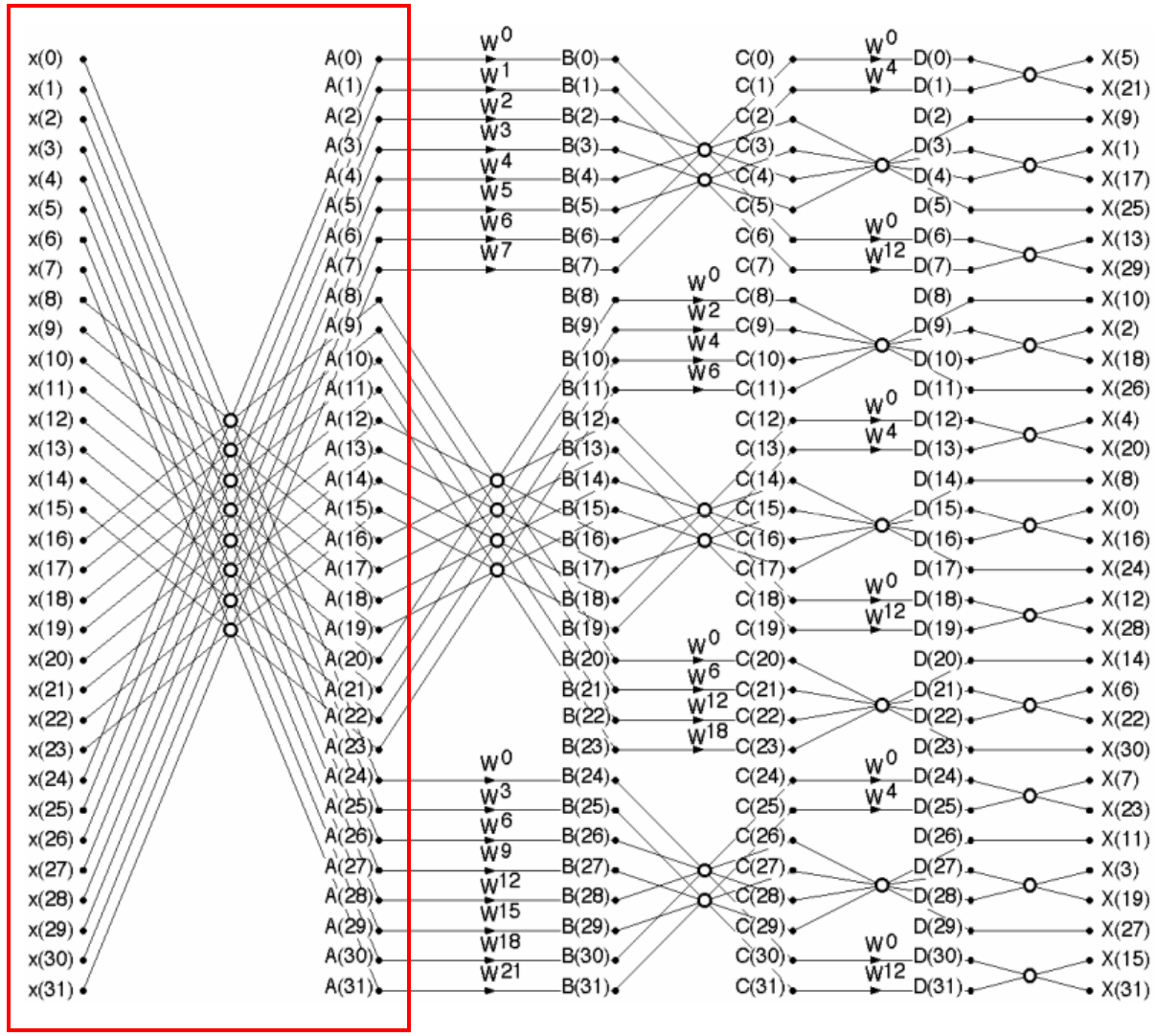
Overview

We will implement **five different** hardware **stages** to accommodate the **complexity** of the **split-radix** algorithm.

The **third stage** is the **scalable** one, while the **others** are designed to handle **exceptions** in the pattern.



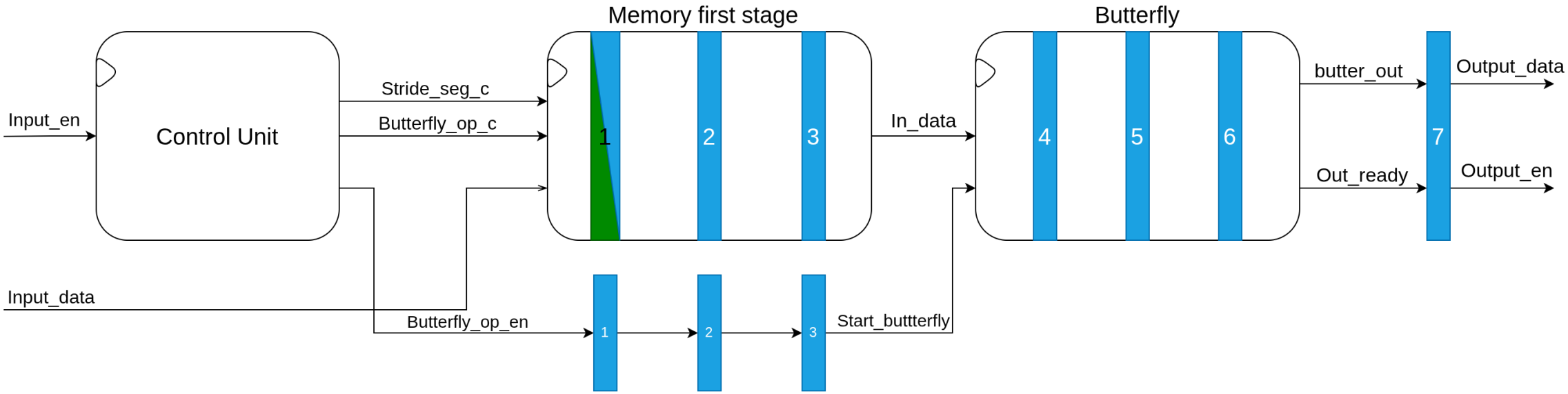
Decomposition into stages



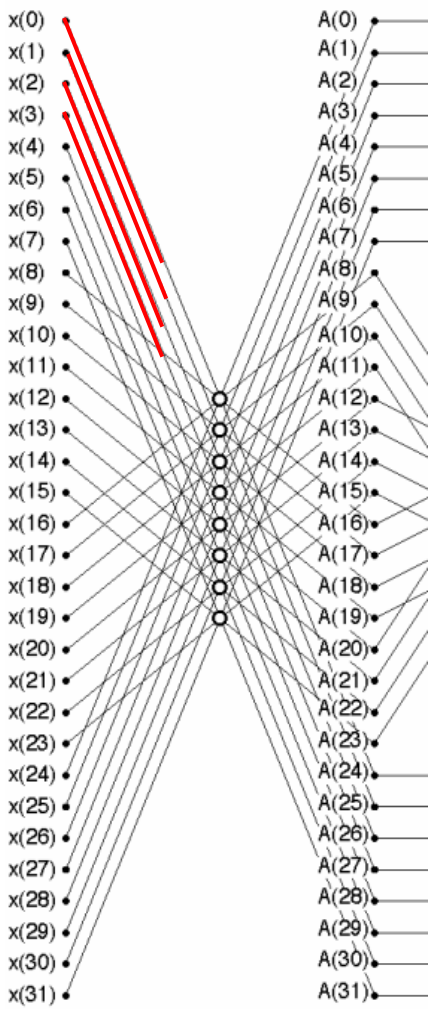
STAGE 1

- **Does not have any multiplications**
- **This structure closely resembles the Radix-4 architecture**

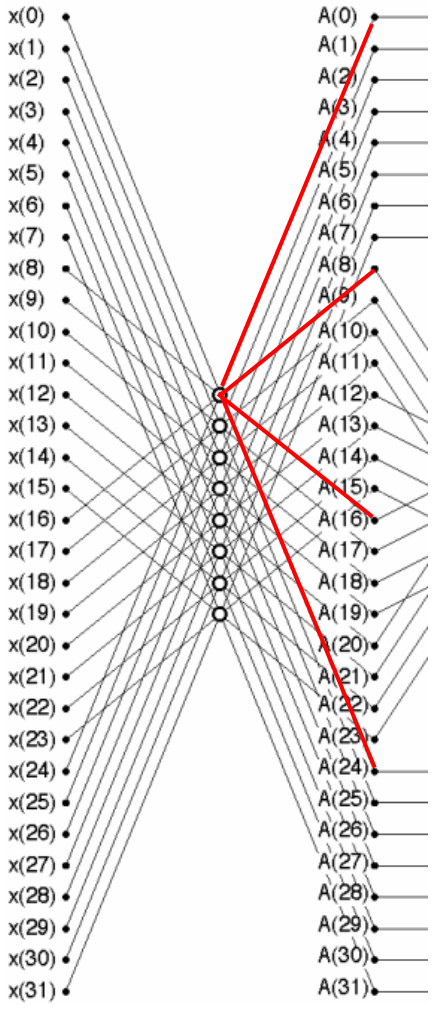
Top diagram 1st stage



Memory pattern 1st stage

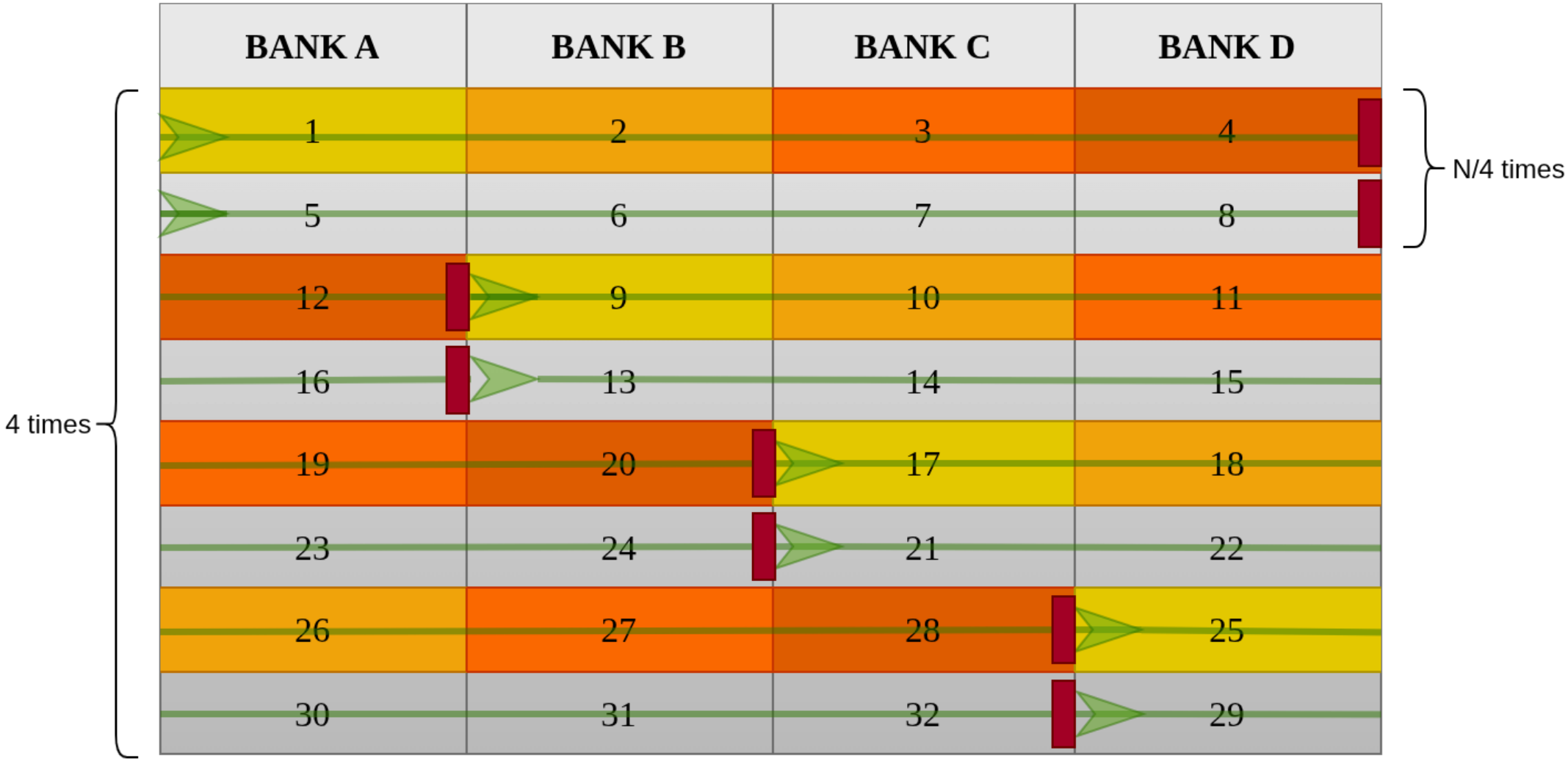


Input patern

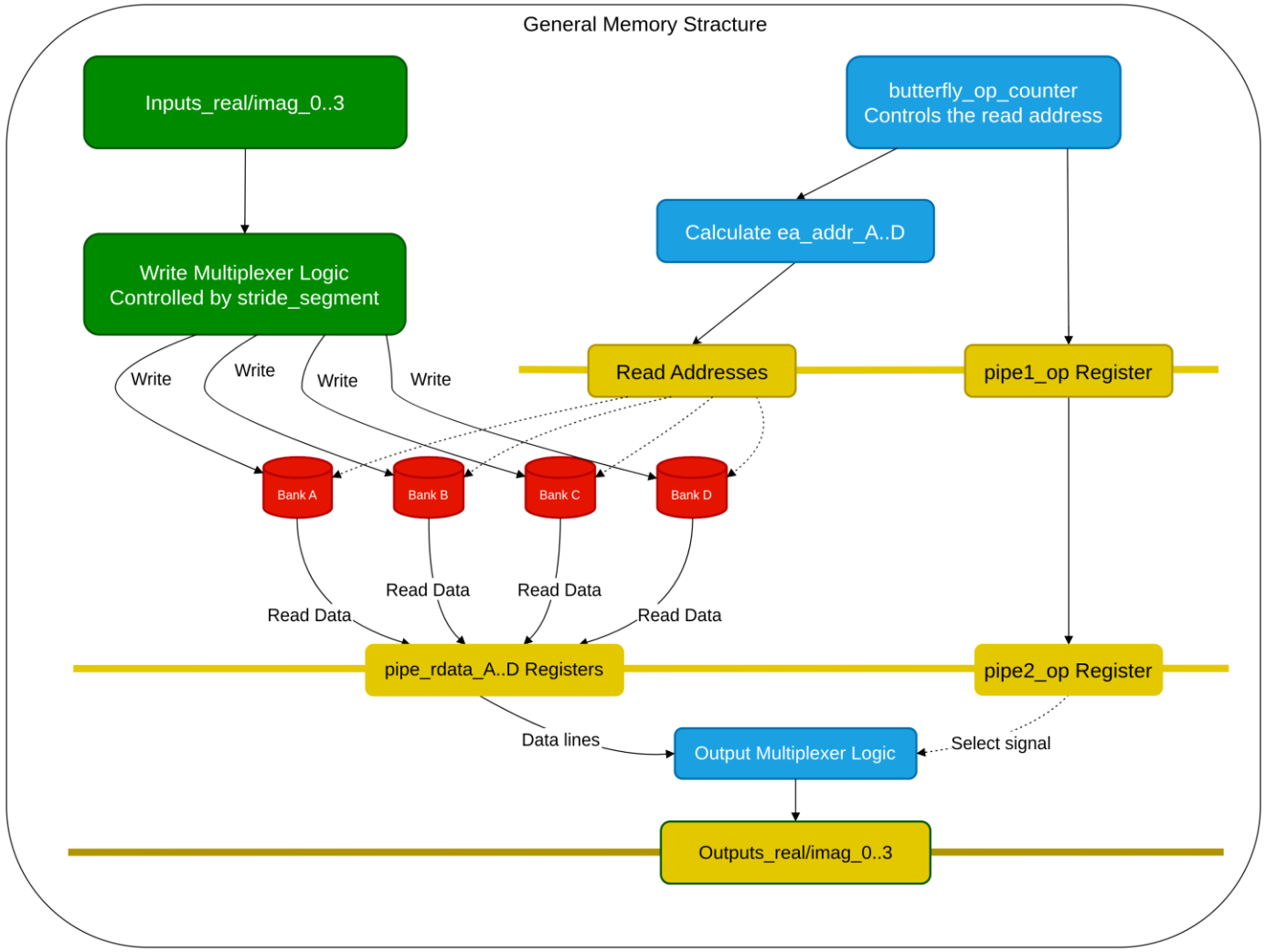


Output patern

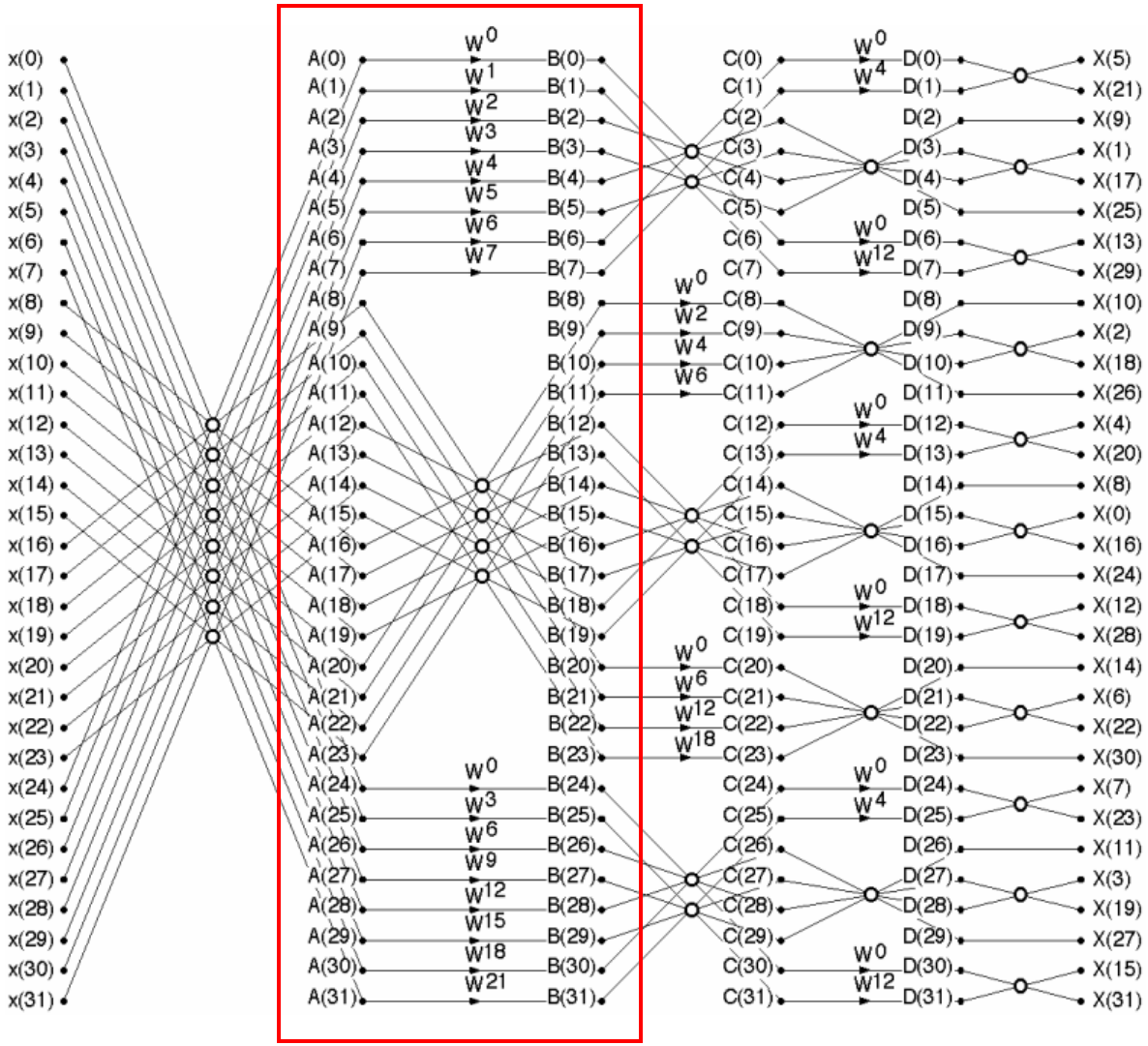
Memory pattern 1st stage



Memory pattern general diagram



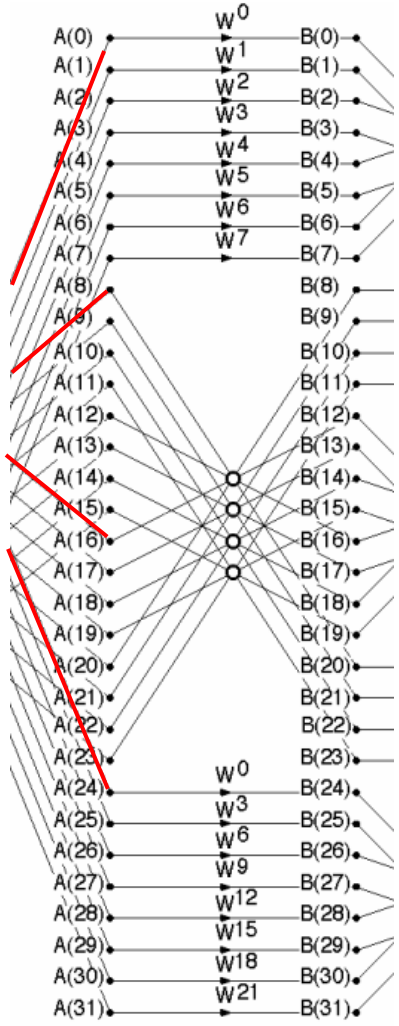
Decomposition into stages



STAGE 2

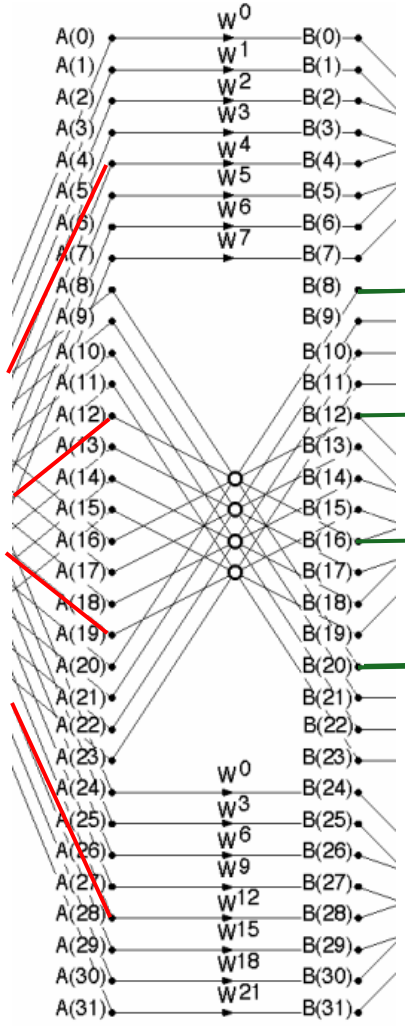
- **First multiplications**
- **New structure/achitecture**

Memory pattern 2st stage



Input patern

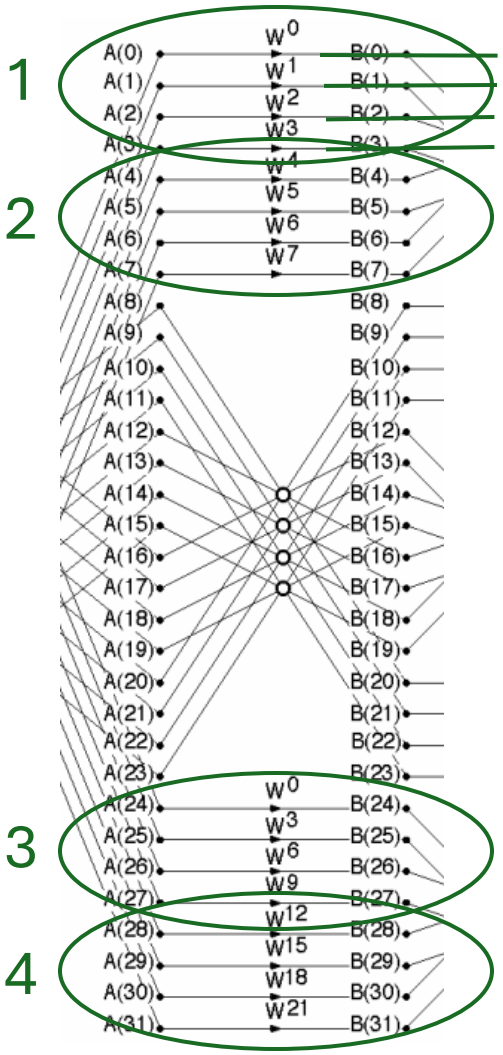
Upper and lower inputs get multiplied before memory



Input patern
Start of output

Middle inputs go to memory for the first half, and then pass thought to the butterfly

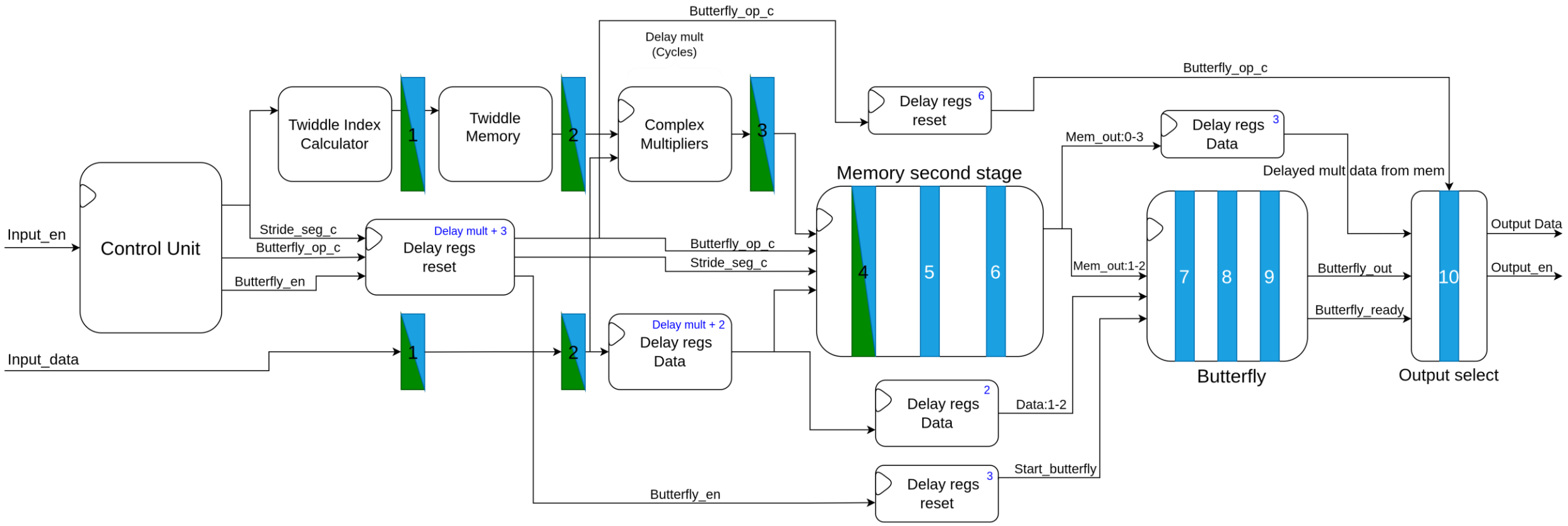
Memory pattern 2st stage



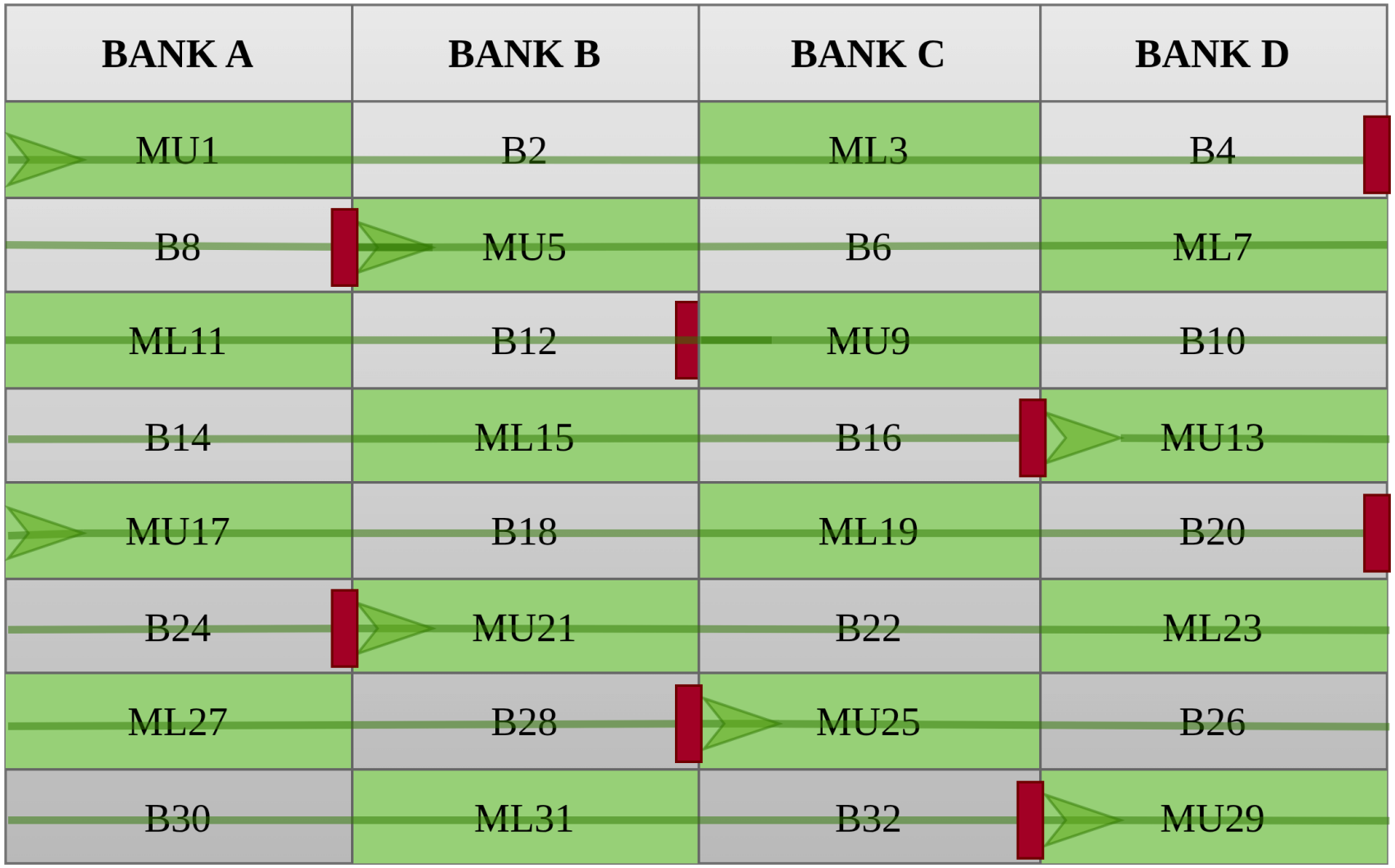
End of output

Memory outputs the stored results of the multiplications

Top diagram 2nd stage

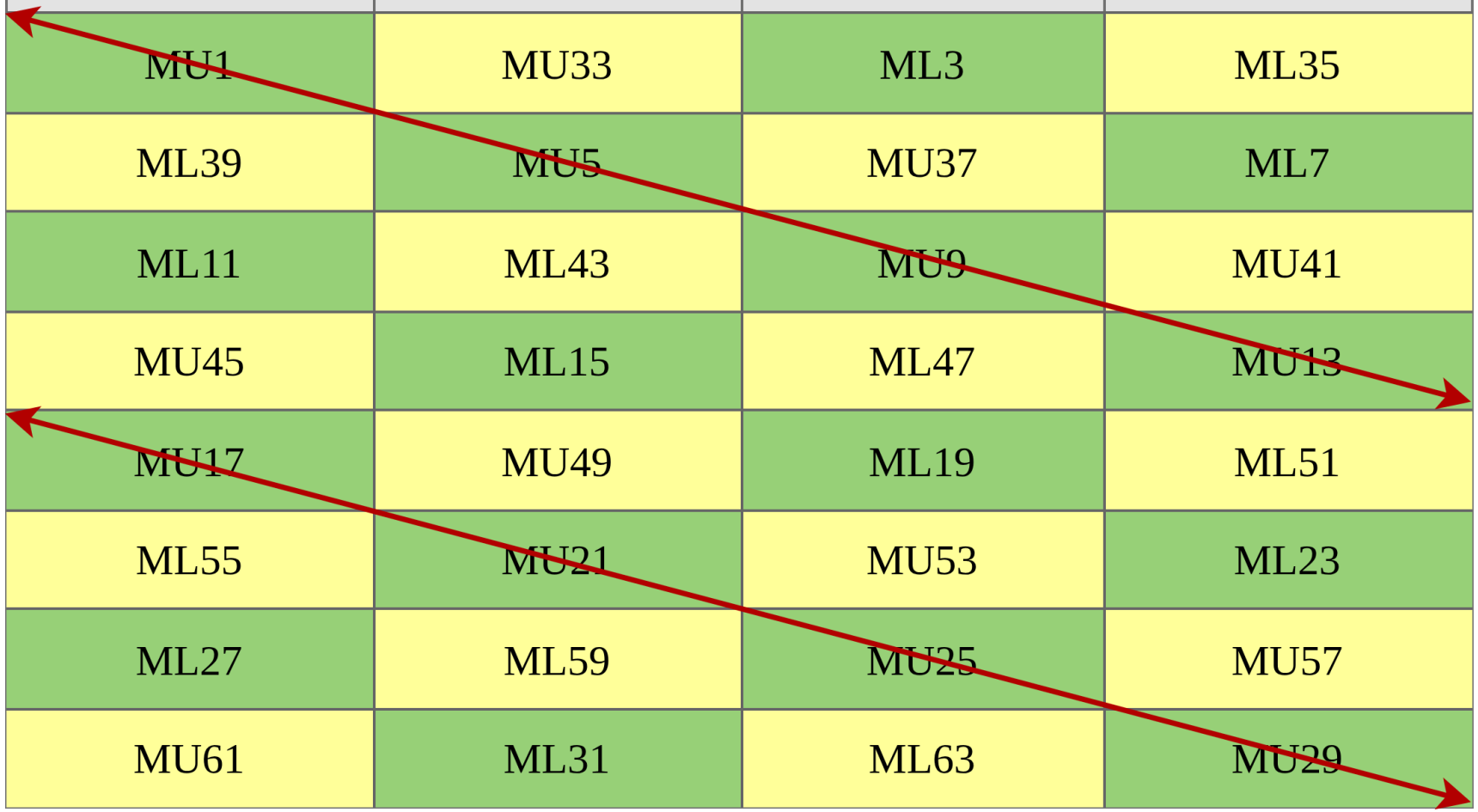


Memory pattern 2nd stage

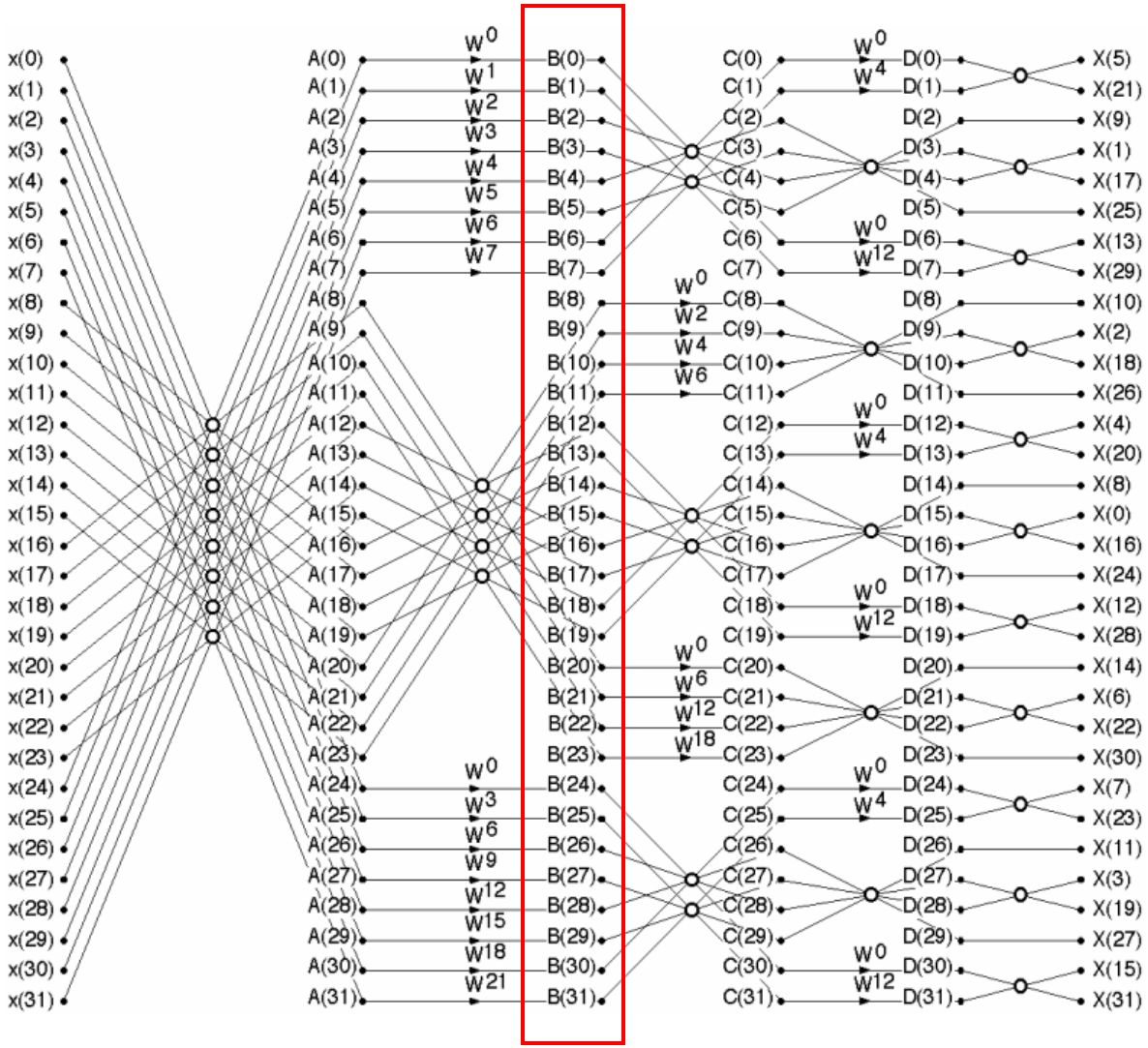


Memory pattern 2nd stage

BANK A	BANK B	BANK C	BANK D
MU1	MU33	ML3	ML35
ML39	MU5	MU37	ML7
ML11	ML43	MU9	MU41
MU45	ML15	ML47	MU13
MU17	MU49	ML19	ML51
ML55	MU21	MU53	ML23
ML27	ML59	MU25	MU57
MU61	ML31	ML63	MU29



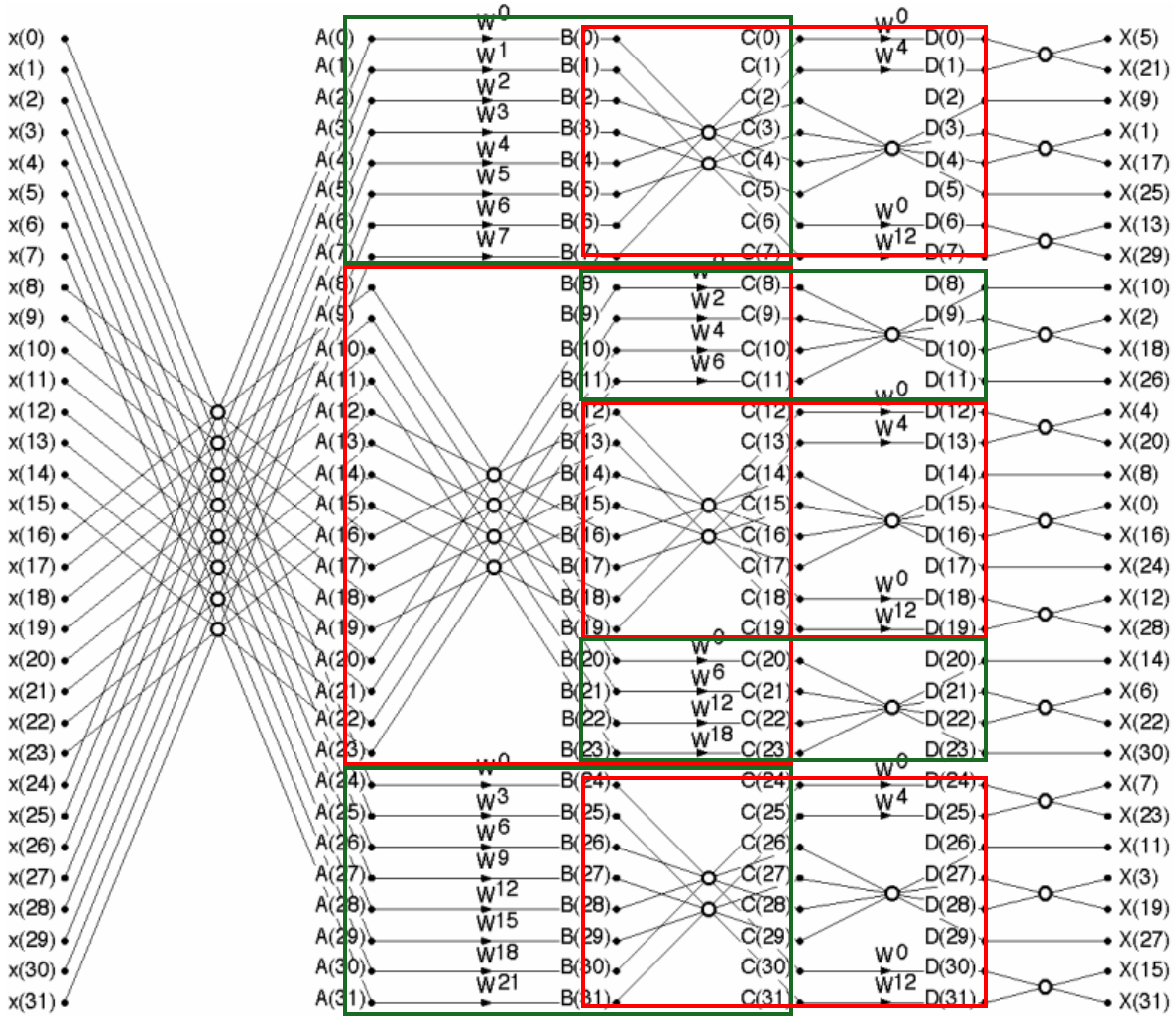
Decomposition into stages



STAGE 3

- **Stege 3 is the stage that can make the design scalable and sits after the first two but before the last two stages**
- **Needs more complex controls, is not visible here as it is needed for ffts larger that 64 samples**

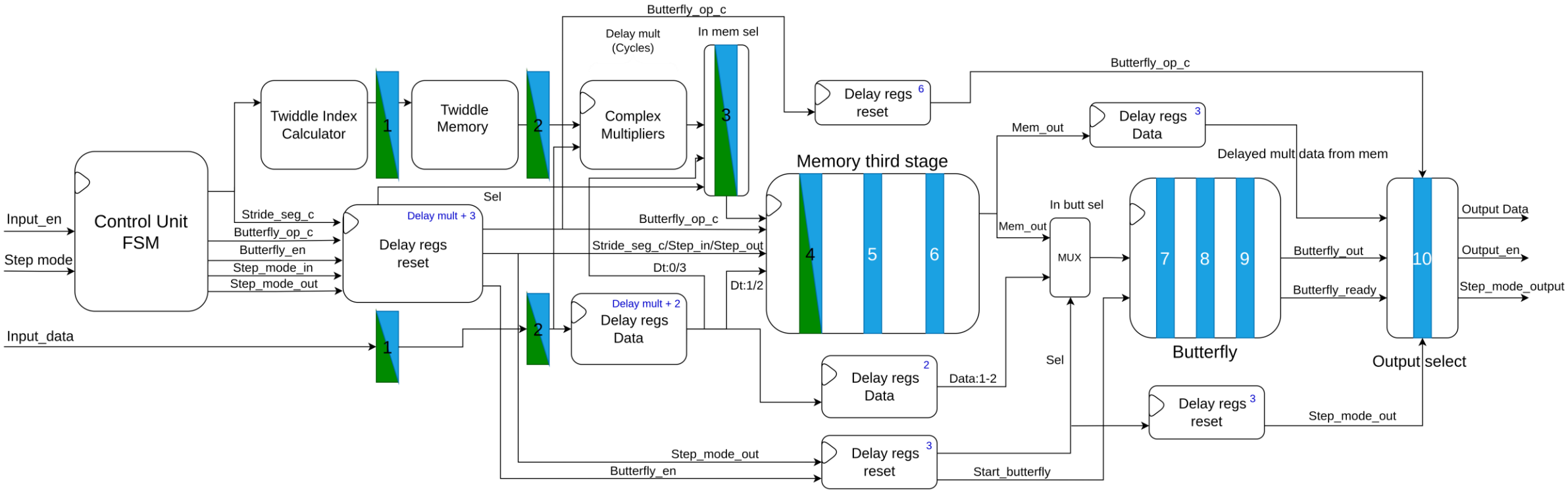
Stage 3 "steps" explained



STAGE 3

- **The red squere indicates step 0**
- **The green ones step 1**
- **The currect's stages step mode is decided by the counters in the control unit plus the stepmode of the previous stage**

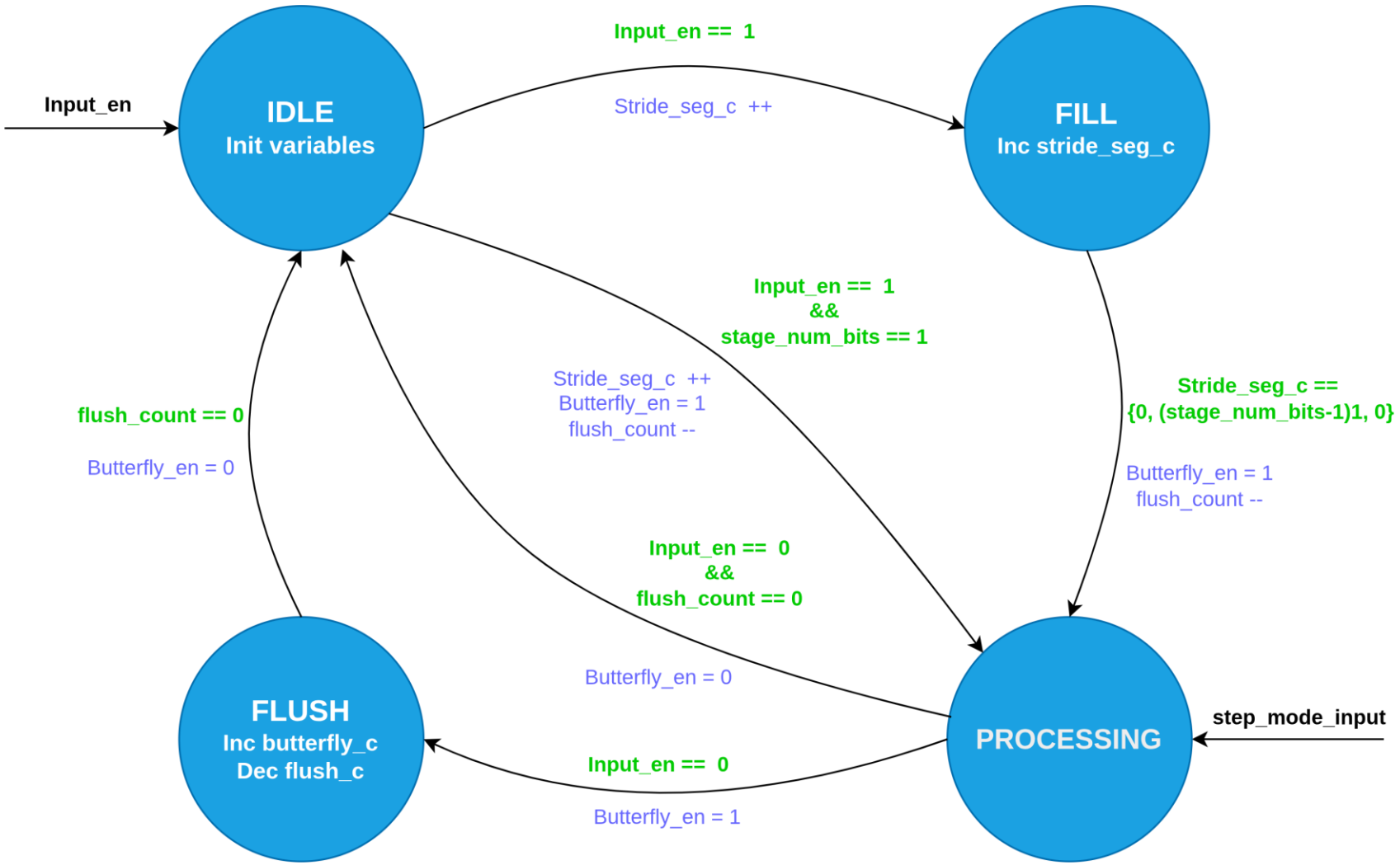
Top diagram 3rd stage



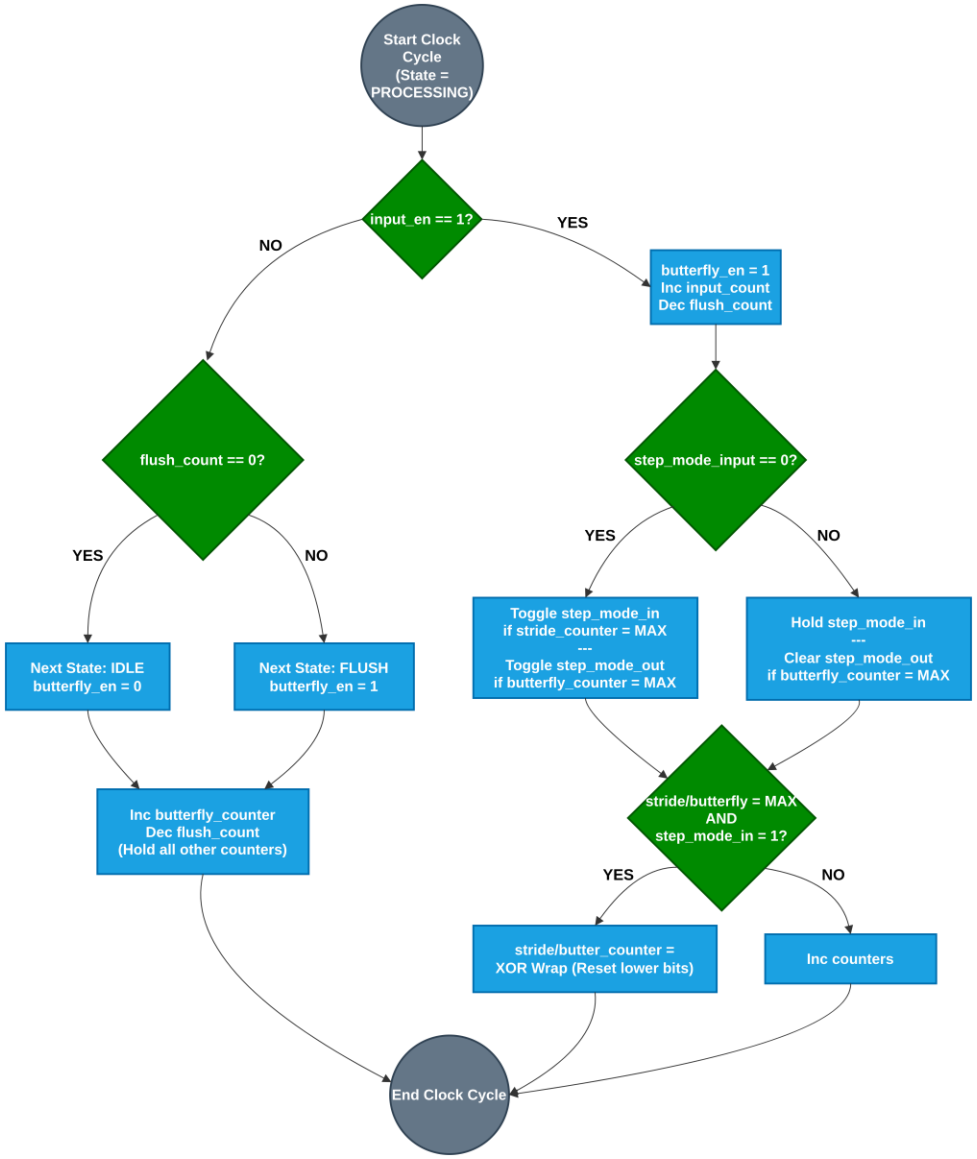
Memory pattern 3rd stage

The **memory** in this stage in a **combination** of the **first** and **second** stage's **memories** since step 0 and step 1 **closely match** stage 1 and 2

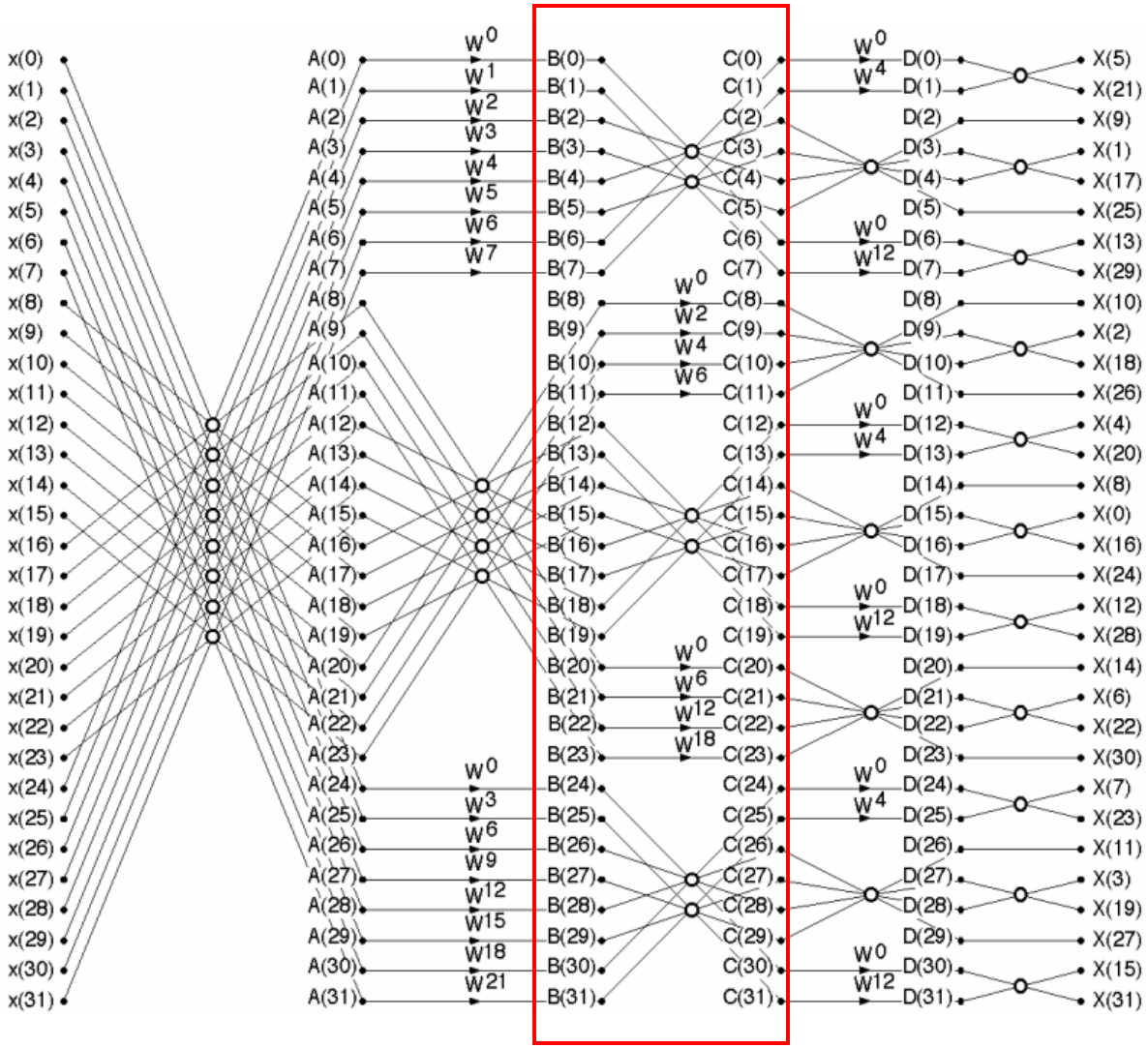
FSM 3rd stage



Processing state diagram



Decomposition into stages

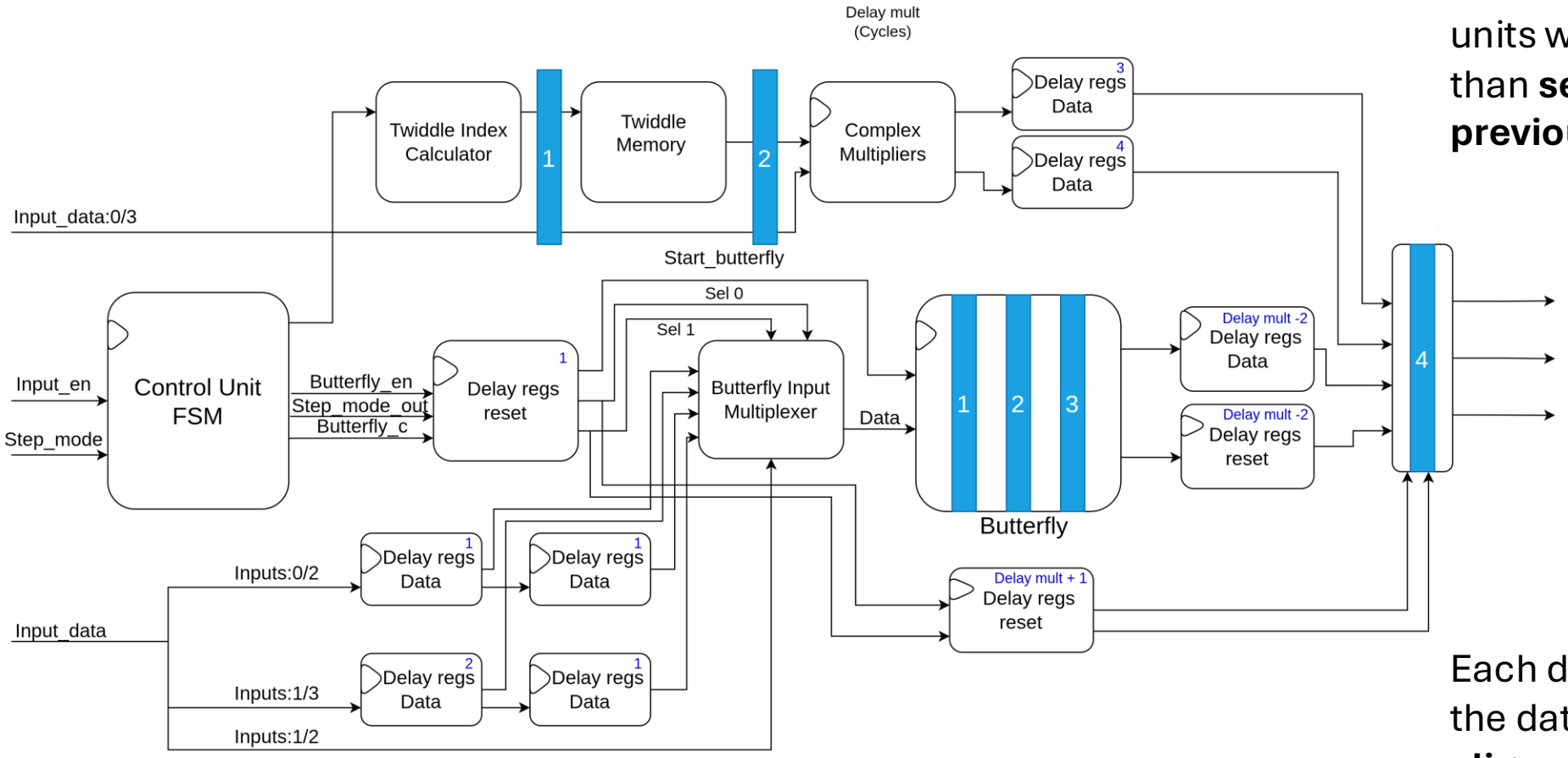


STAGE 4

(second to last)

- **The same exact logic as stage 3 uses the same FSM but the memory is not compatible**
- **For this stage i implement the dealys with shift registers instead of memory that is compatible with bram , since the delays are fixed and small**

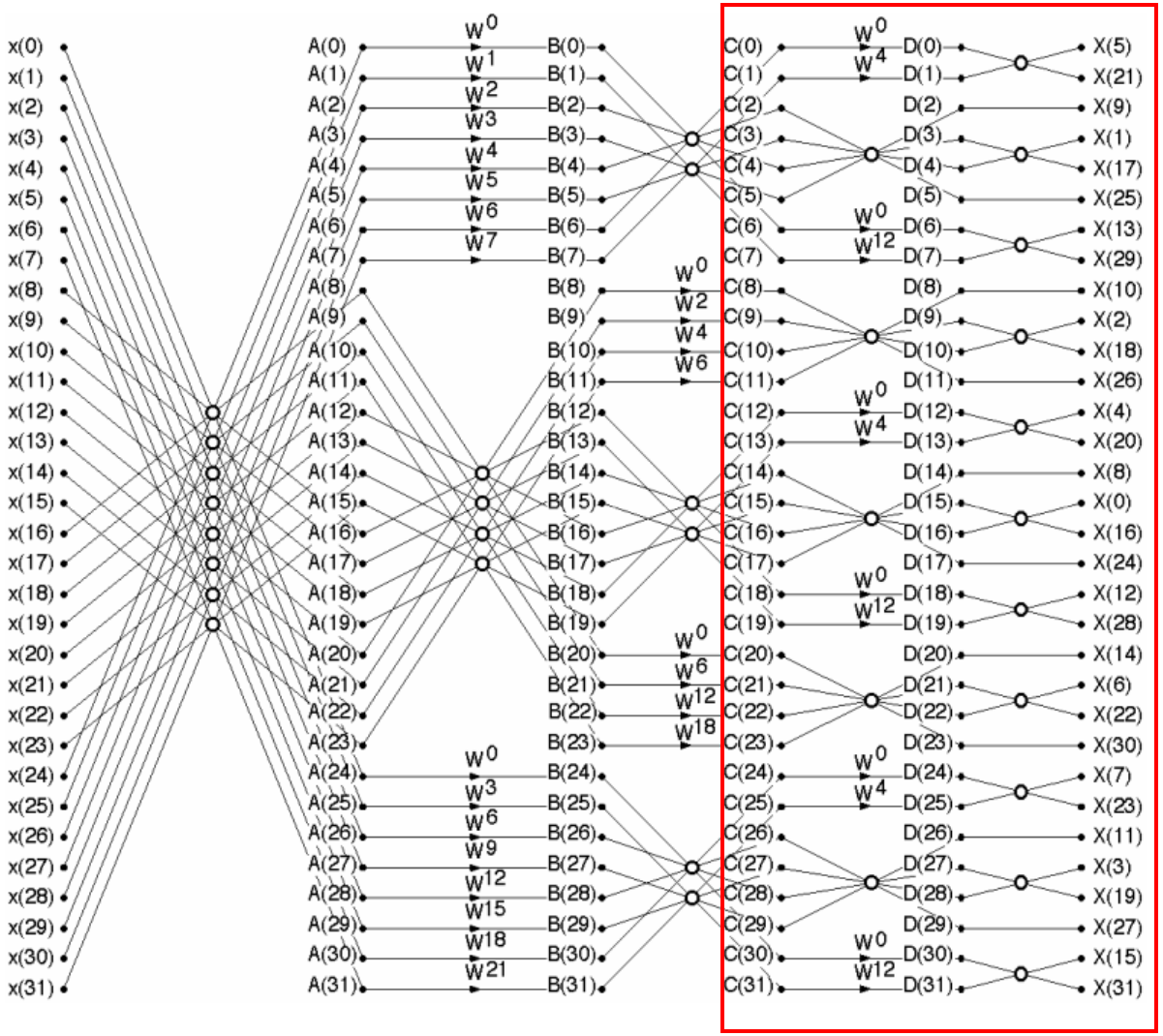
Top diagram 4th stage



In this design, we can see that the **multipliers** and **butterfly** units work in **parallel**, rather than **sequentially** as in the **previous** stages.

Each delay cycle is **exactly** what the data needs to be correctly **aligned** at the **butterfly** input and the stage **output**.

Decomposition into stages

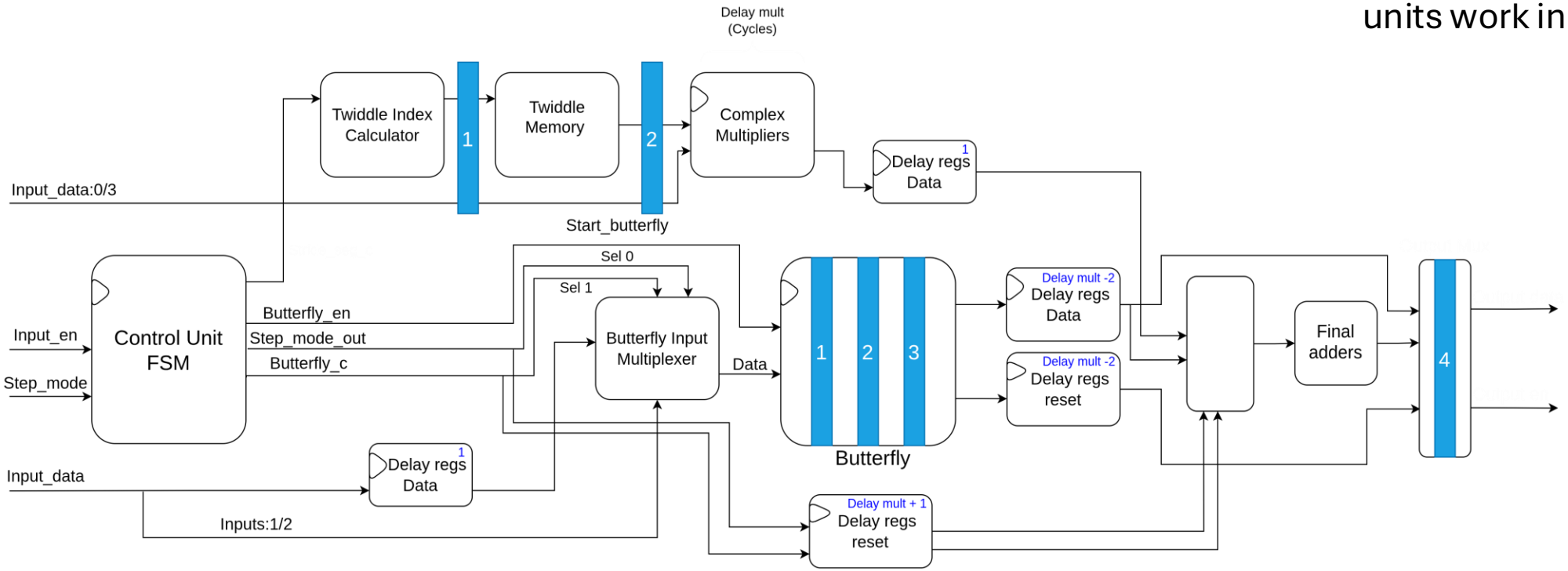


STAGE 5

(last)

- **The same exact logic as stage 3 and 4 uses a simpler FSM**
- **In this stage i combine the last two stages of the algorythm since the last stage is very simple (only one +/-)**

Top diagram 5th stage



In this design, we can see that the **multipliers** and **butterfly** units work in **parallel** again.

At the end there is an extra **adder** for the **last stage** of the algorithm

Two types of complex multiplication modes

Simple multication

Uses 8 mutlipliers

```
always@(posedge clock) begin
    rr_a <= a_re * w0re;
    ii_a <= a_im * w0im;
    ri_a <= a_re * w0im;
    ir_a <= a_im * w0re;

    rr_b <= b_re * w1re;
    ii_b <= b_im * w1im;
    ri_b <= b_re * w1im;
    ir_b <= b_im * w1re;

    out_a_re <= rr_a - ii_a;
    out_a_im <= ri_a + ir_a;
    out_b_re <= rr_b - ii_b;
    out_b_im <= ri_b + ir_b;
end
```

Cheap multication

Uses 6 multipliers

```
always @(posedge clock) begin
    sum_a_in <= a_re + a_im;
    sum_w0_in <= w0re + w0im;
    diff_w0_in <= w0im - w0re;
    sum_b_in <= b_re + b_im;
    sum_w1_in <= w1re + w1im;
    diff_w1_in <= w1im - w1re;
    a_re_d <= a_re;
    a_im_d <= a_im;
    w0re_d <= w0re;
    b_re_d <= b_re;
    b_im_d <= b_im;
    w1re_d <= w1re;

    k1_a <= w0re_d * sum_a_in;
    k2_a <= a_re_d * diff_w0_in;
    k3_a <= a_im_d * sum_w0_in;

    k1_b <= w1re_d * sum_b_in;
    k2_b <= b_re_d * diff_w1_in;
    k3_b <= b_im_d * sum_w1_in;

    out_a_re <= k1_a - k3_a;
    out_a_im <= k1_a + k2_a;

    out_b_re <= k1_b - k3_b;
    out_b_im <= k1_b + k2_b;
end
```

Based on **Gauss's** complex **multiplication**

Results

Table 1: Synthesis Results for 256-point Split-Radix FFT

Design	LUTs	Registers	Clock (ns)	BRAMs	DSPs
Cheap Mult	6586	7518	10.00	0	36
Simple Mult	6581	7004	10.00	0	48

The 'Cheap Mult' version utilizes fewer DSPs however, it requires more registers due to the additional pipeline that the cheap mult module needs.